LINKED LIST

1. Traversal Pattern

When?

- Count nodes
- Search an element
- Print linked list

Template

```
ListNode* temp = head;

while(temp){
    // process node
    temp = temp->next;
}
```

Questions

- Linked List Elements
- Search in Linked List

```
class Solution {
  public:
    bool searchKey(Node* head, int key) {
        Node* temp=head;
        while(temp){
            if(temp->data==key){
                return true;
            }
            temp=temp->next;
        }
        return false;

    }
};
```

- Length of Linked List

```
  public:
    int getCount(Node* head) {
        Node* temp=head;
        int count=0;
        while(temp){
            count++;
            temp=temp->next;
        }
        return count;

    }
};
```

---

2. Fast & Slow Pointer Pattern ★ ★ ★

When?

- Find middle

- Detect cycle
- Palindrome
- Reorder list

Template

```
ListNode* slow = head;
ListNode* fast = head;

while(fast && fast->next){
    slow = slow->next;
    fast = fast->next->next;
}
```

Questions

- Middle of Linked List

```
public:
    ListNode* middleNode(ListNode* head) {
        ListNode* slow=head;
         ListNode* fast=head;
        while(fast&&fast->next){
            slow=slow->next;
            fast=fast->next->next;
        }
        return slow;


    }
};
```

- Linked List Cycle

```

// slow=0-(3), 1(2), 2(0), 3(4)
//fast=0-(3), 2(0), 1(2) ,3(4)
class Solution {
public:
    bool hasCycle(ListNode *head) {
        ListNode* slow=head;
        ListNode* fast=head;
        while(fast&&fast->next){
            slow=slow->next;
            fast=fast->next->next;
            if(slow==fast) return true;
        }
        return false;
    }
};
```

- Linked List Cycle II

```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        ListNode* slow=head;
        ListNode* fast=head;
        while(fast&&fast->next){
            slow=slow->next;
            fast=fast->next->next;
           // temp=s
            if(slow==fast){
                ListNode* temp=head;
                while(temp!=slow){
                    temp=temp->next;
                    slow=slow->next;
                }
                return temp;
            }
        }
        return NULL;

```

- Palindrome Linked List

```
bool isPalindrome(ListNode* head) {
    ListNode* temp=head;
    stack<int>s;
    while(temp){
        s.push(temp->val);
        temp=temp->next;
    }
    while(head){
        int c=s.top();
        s.pop();
        if(head->val!=c){
            return false;
        }
        head=head->next;
    }
    return true;

}
```

- Reorder List

```
public:
    void reorderList(ListNode* head) {
        ListNode* temp1=head;
          ListNode* temp2=head;
        while(temp2&&temp2->next){
            temp1=temp1->next;
            temp2=temp2->next->next;
        }
        ListNode* curr=temp1->next;
        temp1->next=nullptr;
//reverse linkedlist
        ListNode* prev=nullptr;
            while (curr) {
            ListNode* nxt = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nxt;
        }
        //merging two link list alternate
        ListNode* slow=head;
        ListNode* fast=prev;
        while(fast){
            ListNode* temp1=slow->next;
            ListNode* temp2=fast->next;
          slow->next=fast;
          fast->next=temp1;
          slow=temp1;
          fast=temp2;
        }

    }
```

3. Reverse Pattern ★ ★ ★

When?

- Reverse entire list

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev=nullptr;
    ListNode* temp=head;
    while(temp){
        ListNode* rev=temp->next;
        temp->next=prev;
        prev=temp;
        temp=rev;
    }
    return prev;
}
```

- Reverse part of list
- Reverse groups

Template

```
ListNode* prev = NULL;
ListNode* curr = head;

while(curr){
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}
```

Questions

- Reverse Linked List
- Reverse Linked List II

```
ListNode* reverseBetween(ListNode* head, int left, int right) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev= &dummy;
    for(int i=1;i<left;i++){//prev=1
        prev=prev->next;
    }
    ListNode* curr=prev->next;//2
    ListNode* pre=nullptr;//prev->null
    for(int i=0;i<=right-left;i++){
        ListNode* nxt=curr->next;
        curr->next=pre;
        pre=curr;
        curr=nxt;
    }//pre:4->3->2->null  //curr:5
    prev->next->next=curr;1->2->5
    prev->next=pre;1->4->3->2->5

    return dummy.next;
}
```

- Reverse Nodes in K Group -hard
- Palindrome Linked List -done
- Reorder List -done

---

4. Two Pointer Gap Pattern ★★★

When?

Find nth node from end

Template

```
fast = head;
slow = head;

for(int i=0;i<n;i++){
    fast = fast->next;
}

while(fast){
    fast = fast->next;
    slow = slow->next;
}
```

Questions

- Remove Nth Node From End

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode* slow=head;
    ListNode* fast=head;
    for(int i=0;i<n;i++){
        fast=fast->next;
    }
    if(fast==nullptr){
        return head->next;
    }
    while(fast->next){
        fast=fast->next;
        slow=slow->next;
    }
    slow->next=slow->next->next;
    return head;
```

- Find Nth Node From End

---

5. Dummy Node Pattern ★★★

When?

Deletion or insertion at head is possible.

Template

```
ListNode* dummy = new ListNode(0);
dummy->next = head;
```

Questions

- Remove Nth Node From End
- Merge Two Sorted Lists

```
public:
  ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
      ListNode dummy(0);
      ListNode* newnode=&dummy;
      ListNode* temp1=list1;
      ListNode* temp2=list2;
      while(temp1&&temp2){
          if(temp1->val<temp2->val){
              newnode->next=temp1;
              temp1=temp1->next;
          }
          else{
              newnode->next=temp2;
              temp2=temp2->next;
          }
          newnode=newnode->next;

      }
      if(temp1) newnode->next=temp1;
       if(temp2) newnode->next=temp2;


      return dummy.next;

  }
```

- Swap Nodes in Pairs

```
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode dummy(0);
        dummy.next = head;
        ListNode* prev = &dummy;
        while(prev->next && prev->next->next){
            ListNode* first=prev->next;//1
            ListNode* second=first->next;//2
                first->next=second->next;1->3
                second->next=first;2->1
            prev->next=second; 0->2->1->3->4
            prev=first;prev=1

        }
        return dummy.next;
```

- Partition List

```
    ListNode* low=new ListNode(0);
    ListNode* high=new ListNode(0);
    ListNode* temp1=low;
    ListNode* temp2=high;
    ListNode* temp=head;
    while(temp){
       if(temp->val<x){
           temp1->next=temp;
           temp1=temp1->next;
       }
       else{
           temp2->next=temp;
           temp2=temp2->next;
       }
      temp=temp->next;
    }
       temp1->next=high->next;
       temp2->next=nullptr;
     return low->next;
}
```

- Delete Duplicates

```
ListNode* deleteDuplicates(ListNode* head)
    ListNode* curr=head;
    while(curr&&curr->next){
        if(curr->val==curr->next->val){
            curr->next=curr->next->next;
        }
        else{
            curr=curr->next;
        }
    }
    return head;

}
```

-

---

6. Merge Pattern ★ ★ ★

When?

Two sorted linked lists

Template

```
while(l1 && l2){
    if(l1->val < l2->val){
        tail->next = l1;
        l1 = l1->next;
    }
    else{
        tail->next = l2;
        l2 = l2->next;
    }
    tail = tail->next;
}
```

Questions

- Merge Two Sorted Lists -
- Merge K Sorted Lists –

```
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<int,vector<int>,greater<int>>pq;
        for(auto x:lists){
            ListNode* temp=x;
            while(temp){
               pq.push(temp->val);
               temp=temp->next;
            }
        }
          ListNode* dummy=new ListNode(0);
        ListNode* newhead=dummy;
        while(!pq.empty()){
            newhead->next=new ListNode(pq.top());
            pq.pop();
            newhead=newhead->next;
        }
        return dummy->next;
```

- Sort List

---

7. Cycle Detection Pattern ★★★

Template

```
slow = head;
fast = head;

while(fast && fast->next){
    slow = slow->next;
    fast = fast->next->next;

    if(slow == fast){
        return true;
    }
}
```

Questions

- Linked List Cycle
- Linked List Cycle II

---

8. Intersection Pattern ★★

Template

```
ListNode* p1 = headA;
ListNode* p2 = headB;
```

```
while(p1 != p2){
    p1 = (p1==NULL)?headB:p1->next;
    p2 = (p2==NULL)?headA:p2->next;
}
```

Questions

- Intersection of Two Linked Lists

---

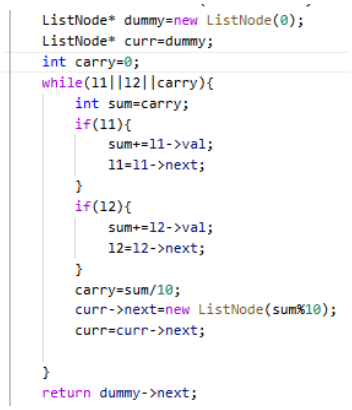
9. Add Numbers Pattern ★ ★

Template

```
int carry = 0;

while(l1 || l2 || carry){
    int sum = carry;

    if(l1){
        sum += l1->val;
        l1 = l1->next;
    }

    if(l2){
        sum += l2->val;
        l2 = l2->next;
    }

    carry = sum/10;
}
```

Questions

- Add Two Numbers



```
ListNode* dummy=new ListNode(0);
ListNode* curr=dummy;
int carry=0;
while(l1||l2||carry){
    int sum=carry;
    if(l1){
        sum+=l1->val;
        l1=l1->next;
    }
    if(l2){
        sum+=l2->val;
        l2=l2->next;
    }
    carry=sum/10;
    curr->next=new ListNode(sum%10);
    curr=curr->next;

}
return dummy->next;
```

- Add Two Numbers II

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    stack<int>st1,st2;
    while(l1){
        st1.push(l1->val);
        l1=l1->next;
    }
     while(l2){
        st2.push(l2->val);
        l2=l2->next;
    }
      ListNode* dummy=new ListNode(0);
    int carry=0;
    while(!st1.empty()||!st2.empty()||carry){
        int sum=carry;
        if(!st1.empty()){
            sum+=st1.top();
            st1.pop();
        }
        if(!st2.empty()){
            sum+=st2.top();
            st2.pop();
        }
        carry=sum/10;
        ListNode* node=new ListNode(sum%10);
        node->next=dummy->next;
        dummy->next=node;
       // without dummy
      // ListNode* node = new ListNode(sum % 10);
      //node->next = head;
    //head = node;

    }
    return dummy->next;

}
```

---

10. Clone / HashMap Pattern ⭐⭐

When?

Copy nodes while maintaining connections.

Questions

- Copy List with Random Pointer

```
public:
    Node* copyRandomList(Node* head) {
        if(head == nullptr)
            return nullptr;
        unordered_map<Node*,Node*>mp;
        Node* newhead=new Node(head->val);
         Node* oldtemp=head->next;
        Node* newtemp=newhead;
        mp[head]=newhead;
        while(oldtemp!=nullptr){
            Node* copynode=new Node(oldtemp->val);
            mp[oldtemp]=copynode;
            newtemp->next=copynode;
            oldtemp=oldtemp->next;
            newtemp=newtemp->next;
        }
        oldtemp=head;
        newtemp=newhead;
        while(oldtemp!=nullptr){
            newtemp->random=mp[oldtemp->random];
            oldtemp=oldtemp->next;
            newtemp=newtemp->next;
        }
        return newhead;

    }
```

---

11. Doubly Linked List Pattern ⋆ ⋆

Questions

- Flatten Multilevel Doubly Linked List
- Browser History
- LRU Cache

Most Important Pattern Ranking

⋆ ⋆ ⋆ Must Master

1. Fast & Slow Pointer
2. Reverse Linked List
3. Dummy Node
4. Merge Lists
5. Two Pointer Gap
6. Cycle Detection

⋆ ⋆ Medium Priority
7. Intersection
8. Add Two Numbers
9. Clone Random Pointer

⋆ ⋆ Advanced
10. Doubly Linked List
11. LRU Cache

Best Practice Order (for you)

1. Reverse Linked List -done
2. Middle of Linked List -done
3. Linked List Cycle
4. Remove Nth Node From End
5. Merge Two Sorted Lists -done
6. Palindrome Linked List -done
7. Intersection of Two Linked Lists -done
8. Reorder List -done
9. Add Two Numbers -done
10. Reverse Linked List II -done
11. Linked List Cycle II-done
12. Merge K Sorted Lists -done
13. Reverse Nodes in K Group

14. Copy List with Random Pointer -done
15. LRU Cache
16. Rotate list-done

Swapping Nodes in a Linked List

```
ublic:
    ListNode* swapNodes(ListNode* head, int k) {
        ListNode* first=head;
        for(int i=1;i<k;i++){
            first=first->next;
        }
        ListNode* temp1=first;
        ListNode* temp2=head;
        while(temp1->next){
            temp1=temp1->next;
            temp2=temp2->next;
        }
        swap(first->val,temp2->val);
        return head;


    `
```

Rotate List

```
public:
    ListNode* rotateRight(ListNode* head, int k) {
        if(head==nullptr) return 0;
        ListNode* tail=head;
        int n=1;
        while(tail->next){
            n++;
            tail=tail->next;
        }
        k%=n;//2%5=2
          if(k == 0)
            return head;
        tail->next=head;//5->1
        int steps=n-k;//5-2=3
        ListNode* newtail=tail;//5
        while(steps--){
            newtail=newtail->next;//5->1->2->3//move three steps//and 3 is newtail
        }
          ListNode* newHead = newtail->next;//newhead->4->5
        newtail->next = nullptr;//3->null

        return newHead;
    }
-   ,,
5   //sabse pehle node count kiaa..
7   //joh ll h uske tail ke next ko head krdo..
3   ///fir k%=n nikal lo..
9   //fir steps nikalo ki n-k kr dege toh pata chal jaega ki kitna aage badhke ll todni h
0   ///aab ek tarah s eyeh ek circlar ll h jab hamne tail ke next ko head econnect kia tha tab circular ban gy athaa toh uske baad
    tail ek tarah se last element h uss tail ko newtail me store kia h ki rotate krne ke baad new tail konsa banega aur joh steps
    nikale the utane steps aage move krva do toh jaha tak move hoga vhi hamara new tail ban jaega ...
1   //aur newtail ke next vala new head...
2   //fir new tail ke next ko null krdo bss fir new head return krdooo
```